

Lesson 14: Conditionals

Learning Objectives

- Use ternary expressions for conditional values
 - Use `count` for conditional resource creation
 - Use `for` expressions to filter lists and maps
 - Combine conditionals with locals for dynamic configuration
-

1. Ternary Expressions

The ternary operator is Terraform's primary conditional construct:

```
condition ? true_value : false_value
```

```
# Basic ternary
bucket_name = var.environment == "prod" ? "prod-data-bucket" :
"${var.environment}-data-bucket"

# Nested ternary
retention = var.environment == "prod" ? 365 : (var.environment == "qa" ? 90
: 7)

# With functions
instance_type = var.environment == "prod" ? "t3.large" : "t3.micro"
```

Ternary in Resource Arguments

```
resource "aws_sqs_queue" "main" {
  name = "${var.environment}-queue"
  message_retention_seconds = var.environment == "prod" ? 1209600 : 345600
  delay_seconds = var.environment == "prod" ? 0 : 5
}
```

Ternary in Tags

```
tags = {
  Environment = var.environment
  IsProduction = var.environment == "prod" ? "true" : "false"
  Versioning   = var.enable_versioning ? "enabled" : "disabled"
}
```

2. Conditional Resource Creation with `count`

The most common pattern for conditional resources:

```
# Create only when condition is true
resource "aws_s3_bucket" "audit" {
  count = var.enable_audit ? 1 : 0
  bucket = "audit-bucket"
}

# Reference with [0] when count = 1
locals {
  audit_bucket_id = var.enable_audit ? aws_s3_bucket.audit[0].id : null
}
```

Conditional with `for_each`

```
resource "aws_sqs_queue" "main" {
  for_each = var.environment == "prod" ? var.prod_queues : var.dev_queues
  name     = "${each.key}-queue"
}
```

3. Conditional Values in Locals

Centralizing conditional logic in `locals` keeps your configuration clean:

```
locals {
  is_prod    = var.environment == "prod"
  is_dev     = var.environment == "dev"

  # Count configuration
  bucket_count = local.is_prod ? 3 : 1

  # Size/tier configuration
  instance_tier = local.is_prod ? "large" : (local.is_dev ? "small" : "medium")
}
```

```
# Feature flags
enable_versioning = !local.is_prod
enable_encryption = var.environment != "legacy"
retention_days    = local.is_prod ? 365 : 7

# Conditional names
bucket_prefix = local.is_prod ? "prod" : "nonprod"
}
```

4. **for** Expressions for Filtering

for expressions create new collections by transforming or filtering existing ones:

```
# Filter a list – keep only items matching condition
locals {
  all_queues = ["orders", "billing", "shipping", "audit"]
  prod_queues = [for q in local.all_queues : q if q != "audit"]
}

# Filter a map
variable "instance_configs" {
  type = map(object({
    enabled = bool
    size    = number
  }))
  default = {
    web    = { enabled = true, size = 2 }
    worker = { enabled = true, size = 5 }
    legacy = { enabled = false, size = 1 }
  }
}

locals {
  enabled_configs = {
    for name, config in var.instance_configs : name => config
    if config.enabled
  }
}

# Transform values in a for expression
locals {
  upper_names = [for q in local.all_queues : upper(q)]
  # Result: ["ORDERS", "BILLING", "SHIPPING", "AUDIT"]
}
```

5. Conditional with **try** and **can**

try evaluates expressions and returns a fallback on error:

```
# Safe access to potentially missing attributes
locals {
  # If the attribute doesn't exist, returns "unknown"
  bucket_region = try(aws_s3_bucket.main.region, "unknown")

  # Nested safe access
  queue_arn = try(aws_sqs_queue.main[0].arn, null)
}

# Combined with count conditional
resource "aws_sqs_queue" "optional" {
  count = var.create_queue ? 1 : 0
  name  = "optional-queue"
}

locals {
  queue_arn = try(aws_sqs_queue.optional[0].arn, null)
}
```

6. Complete Example

```
# variables.tf
variable "environment" {
  description = "Deployment environment"
  type        = string
  default     = "dev"
}

variable "enable_audit" {
  description = "Whether to create audit infrastructure"
  type        = bool
  default     = false
}

# locals.tf
locals {
  is_prod      = var.environment == "prod"
  is_dev       = var.environment == "dev"
  enable_audit = var.enable_audit || local.is_prod
  retention_days = local.is_prod ? 365 : 7
  enable_versioning = !local.is_prod
  audit_buckets    = local.enable_audit ? ["audit-logs", "audit-backup"] : []
}

# Filtered queue list
all_queues = concat(
  ["orders", "billing"],
  local.enable_audit ? ["audit-events"] : []
)
```

```
}

# main.tf
# Conditional bucket via count
resource "aws_s3_bucket" "data" {
  bucket = "${var.environment}-data-bucket"
}

# Conditional versioning via ternary
resource "aws_s3_bucket_versioning" "data" {
  count = local.enable_versioning ? 1 : 0
  bucket = aws_s3_bucket.data.id

  versioning_configuration {
    status = "Enabled"
  }
}

# Audit bucket via count
resource "aws_s3_bucket" "audit" {
  count = local.enable_audit ? 1 : 0
  bucket = "${var.environment}-audit-bucket"
}

# Queues from filtered list
resource "aws_sqs_queue" "main" {
  for_each = toset(local.all_queues)
  name     = "${each.key}-${var.environment}"
}

# Conditionally set retention via ternary
resource "aws_sqs_queue" "events" {
  name = "${var.environment}-events"
  message_retention_seconds = local.retention_days * 86400
}

# outputs.tf
output "bucket_name" {
  value = aws_s3_bucket.data.id
}

output "audit_bucket" {
  value = try(aws_s3_bucket.audit[0].id, null)
}

output "queues" {
  value = [for q in aws_sqs_queue.main : q.name]
}

output "versioning_enabled" {
  value = local.enable_versioning
}
```

7. Key Takeaways

- **Ternary** (`condition ? true : false`) is the primary conditional for values
- **count with ? 1 : 0** conditionally creates or skips resources
- **for expressions** filter lists and maps with `if` clauses
- **try()** provides safe access to potentially missing attributes (useful with conditional resources)
- Centralize conditional logic in **locals** to keep resources clean
- Conditional resource references need `[0]` access and `try()` for safety
- Three patterns cover most needs: ternary for values, count for resources, for for collections

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer - Eyuel M.